

Practical: Fashion Clothing Classification using CNN (MNIST Fashion Dataset)

Laboratory Practice V – Computer Engineering

1 Aim

Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories using Convolutional Neural Network (CNN).

1.1 Objective

Students should be able to use MNIST Fashion Dataset and create a classifier to classify fashion clothing into 10 categories.

2 Theory

2.1 What is Classification?

Classification is a type of **supervised learning** that involves categorizing data into predefined classes based on features. A model is trained on labeled data, learns the patterns, and then classifies new, unseen data.

2.2 What is CNN (Convolutional Neural Network)?

CNN is a type of Deep Neural Network specially designed for **image processing**. It automatically learns features like edges, textures, and patterns from images.

2.2.1 Key Layers in CNN

Layer	Purpose
Conv2D	Applies filters to extract features from images
MaxPooling2D	Reduces spatial dimensions by taking max value in each region
Dropout	Randomly disables neurons to prevent overfitting
Flatten	Converts 2D feature maps to 1D vector
Dense	Fully connected layer for classification
Softmax	Converts output to probability distribution over classes

2.3 MNIST Fashion Dataset

Property	Details
Total Images	70,000 grayscale images (28×28 pixels)
Training Set	60,000 images
Test Set	10,000 images
Classes	10 categories
Source	Zalando Research (2017)

2.3.1 10 Classes

Label	Category	Label	Category
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

3 Code

Listing 1: Fashion MNIST Classification using CNN

```

1 # Step 1: Import Libraries & Load Dataset
2 import tensorflow as tf
3 import matplotlib.pyplot as plt
4 from tensorflow import keras
5 import numpy as np
6
7 (x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.
   load_data()
8
9 # Step 2: Visualize Sample Images
10 plt.imshow(x_train[0])
11 plt.show()
12
13 # Step 3: Preprocess Data
14 x_train = x_train.astype('float32') / 255.0
15 x_test = x_test.astype('float32') / 255.0
16
17 x_train = x_train.reshape(-1, 28, 28, 1)
18 x_test = x_test.reshape(-1, 28, 28, 1)
19
20 # Step 4: Build CNN Model
21 model = keras.Sequential([
22     # Conv Layer 1: 32 filters, 3x3
23     keras.layers.Conv2D(32, (3,3), activation='relu',
24                          input_shape=(28,28,1)),
25     keras.layers.MaxPooling2D((2,2)),
26     keras.layers.Dropout(0.25),
27
28     # Conv Layer 2: 64 filters, 3x3
29     keras.layers.Conv2D(64, (3,3), activation='relu'),
30     keras.layers.MaxPooling2D((2,2)),

```

```

31     keras.layers.Dropout(0.25),
32
33     # Conv Layer 3: 128 filters, 3x3
34     keras.layers.Conv2D(128, (3,3), activation='relu'),
35
36     # Flatten + Dense layers
37     keras.layers.Flatten(),
38     keras.layers.Dense(128, activation='relu'),
39     keras.layers.Dropout(0.25),
40     keras.layers.Dense(10, activation='softmax')
41 ])
42
43 model.summary()
44
45 # Step 5: Compile and Train
46 model.compile(optimizer='adam',
47               loss='sparse_categorical_crossentropy',
48               metrics=['accuracy'])
49
50 history = model.fit(x_train, y_train, epochs=10,
51                    validation_data=(x_test, y_test))
52
53 # Step 6: Evaluate
54 test_loss, test_acc = model.evaluate(x_test, y_test)
55 print('Test accuracy:', test_acc)

```

4 Output

4.1 Model Summary

```

Model: "sequential"
+-----+-----+-----+
| Layer (type)          | Output Shape      | Param # |
+-----+-----+-----+
| conv2d (Conv2D)        | (None, 26, 26, 32)|      320 |
| max_pooling2d          | (None, 13, 13, 32)|         0 |
| dropout (Dropout)      | (None, 13, 13, 32)|         0 |
| conv2d_1 (Conv2D)      | (None, 11, 11, 64)|    18,496 |
| max_pooling2d_1        | (None, 5, 5, 64)  |         0 |
| dropout_1 (Dropout)    | (None, 5, 5, 64)  |         0 |
| conv2d_2 (Conv2D)      | (None, 3, 3, 128) |    73,856 |
| flatten (Flatten)      | (None, 1152)       |         0 |
| dense (Dense)          | (None, 128)        |    147,584 |
| dropout_2 (Dropout)    | (None, 128)        |         0 |
| dense_1 (Dense)        | (None, 10)         |     1,290 |
+-----+-----+-----+
Total params: 241,546

```

4.2 Training Output (Selected Epochs)

```
Epoch 1/10 - accuracy: 0.7925 - val_accuracy: 0.8508
Epoch 4/10 - accuracy: 0.8930 - val_accuracy: 0.8997
Epoch 7/10 - accuracy: 0.9070 - val_accuracy: 0.9096
Epoch 10/10 - accuracy: 0.9145 - val_accuracy: 0.9078
```

4.3 Final Test Accuracy

```
Test accuracy: 0.9031 (90.31%)
```

5 Code Explanation (Hinglish)

5.1 Is Practical Ka Aim Kya Hai?

Humein MNIST Fashion Dataset ki **70,000 grayscale images** (kapdon ki photos — T-shirt, Trouser, Sneaker, etc.) dekhke batana hai ki image kis **category** ki hai. Total **10 categories** hain.

Yeh ek **Multi-class Image Classification** problem hai — CNN (Convolutional Neural Network) use karke solve karenge.

5.2 Karna Kya Hai? (Har Cell Ka Matlab)

5.2.1 Step 1: Import Libraries & Load Dataset

```
import tensorflow as tf
from tensorflow import keras
(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.
    load_data()
```

- Keras ke built-in function se Fashion MNIST dataset directly download hota hai
- `x_train` = 60,000 training images (28×28 pixels each)
- `y_train` = unke labels (0–9)
- `x_test` = 10,000 test images
- `y_test` = unke labels

Simple: Dataset load kiya — 60K images train ke liye, 10K test ke liye.

5.2.2 Step 2: Visualize Sample Images

```
plt.imshow(x_train[0])
```

Sample images dekhi — ek ankle boot aur ek trouser dikhti hai. Yeh verify karne ke liye ki data sahi load hua hai.

Simple: Dekha ki images kaisi dikhti hain.

5.2.3 Step 3: Preprocess Data

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

- / 255.0 — pixel values ko 0–255 se 0–1 range mein normalize kiya
- reshape(-1, 28, 28, 1) — CNN ko 4D input chahiye: (samples, height, width, channels)
- 1 channel kyunki grayscale hai (RGB hota toh 3)

Simple: Images ko normalize kiya aur CNN ke liye sahi shape mein convert kiya.

5.2.4 Step 4: Build CNN Model

```
model = keras.Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Dropout(0.25),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Dropout(0.25),
    Conv2D(128, (3,3), activation='relu'),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.25),
    Dense(10, activation='softmax')
])
```

Layer by layer samjho:

1. **Conv2D(32, 3×3, relu)** — 32 filters se basic features nikalte hain (edges, lines). Parameters: $32 \times 3 \times 3 + 32 = 320$
2. **MaxPooling2D(2×2)** — image size half: $28 \rightarrow 14$
3. **Dropout(0.25)** — 25% neurons randomly band — overfitting rokta hai
4. **Conv2D(64, 3×3, relu)** — 64 filters se complex features. Parameters: $64 \times 3 \times 3 \times 14 + 64 = 18,496$
5. **MaxPooling2D** — size: $14 \rightarrow 7$
6. **Dropout(0.25)** — phir se 25% dropout
7. **Conv2D(128, 3×3, relu)** — 128 filters se advanced features. Parameters: $128 \times 3 \times 3 \times 7 + 128 = 73,856$
8. **Flatten** — $7 \times 7 \times 128 = 1,152$ values ko ek line mein rakha
9. **Dense(128, relu)** — classification ke liye 128 neurons. Parameters: $1152 \times 128 + 128 = 147,584$

10. **Dropout(0.25)** — 25% dropout

11. **Dense(10, softmax)** — 10 classes ke liye output — har class ki probability

Total parameters: 2,41,546 — yeh sab weights hain jo model training mein seekhega.

Simple: Image → 3 baar (filters se features nikalo + size chhoti karo + dropout) → flatten → classify → 10 mein se kaunsa kapda hai batao.

5.2.5 Step 5: Compile & Train

```
model.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])
history = model.fit(x_train, y_train, epochs=10,
                    validation_data=(x_test, y_test))
```

- **adam** — optimizer jo weights update karega
- **sparse_categorical_crossentropy** — multi-class classification ke liye loss (labels integer format mein hain, one-hot nahi)
- **epochs=10** — dataset 10 baar dekhega
- **batch.size=32** (default) — $60,000/32 = 1,875$ batches per epoch
- **Accuracy:** 79% (Epoch 1) → 91.4% (Epoch 10)
- **Validation:** 85% → 90.8%

Simple: Model ne 10 baar poora dataset dekhke seekha. 91% accuracy mili!

5.2.6 Step 6: Evaluate

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print('Test accuracy:', test_acc) # 90.31%
```

Final test accuracy: 90.31% — 10 mein se ~9 images sahi classify kiye!

Simple: Final exam diya — 90.31% marks aaye.

5.3 Final Outcome Kya Milta Hai?

1. **Trained CNN Model** — jo kapde ki image dekhke 10 mein se category bata sakta hai
2. **90.31% Test Accuracy** — 10 mein se ~9 images sahi classify
3. **Training progression** — 79% se 91% tak improve hua over 10 epochs
4. **Dropout effect** — training aur validation accuracy close hain (kam overfitting)

Bottom Line: Is practical mein humne dekha ki CNN kaise grayscale images (28×28) se automatically features seekhta hai aur 10 fashion categories mein classify karta hai. 3 Conv layers + Dropout + Dense layers ka use karke 90%+ accuracy achieve ki.

6 How to Run

6.1 On Google Colab (Recommended)

1. Go to `colab.research.google.com`
2. Upload the `.ipynb` notebook
3. Runtime → Change runtime type → **T4 GPU**
4. Run all cells — dataset automatically downloads

6.2 On Fresh Ubuntu PC

1. Update & install Python:

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3 python3-pip python3-venv -y
```

2. Create virtual env & install:

```
python3 -m venv dl_env
source dl_env/bin/activate
pip install tensorflow matplotlib numpy jupyter
```

3. Run notebook:

```
jupyter notebook
# Open .ipynb and run all cells
# Dataset downloads automatically via Keras
```

7 FAQ

1. **What is MNIST dataset for classification?**

MNIST Fashion is a collection of 70,000 grayscale images (28×28 pixels) of 10 categories of clothing. It is used as a benchmark for image classification.

2. **How many classes are in the MNIST Fashion dataset?**

10 classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

3. **What is 784 in MNIST dataset?**

Each image is $28 \times 28 = 784$ pixels. When flattened, each image becomes a vector of 784 values.

4. **How many epochs are used?**

10 epochs. More epochs can improve accuracy but may cause overfitting.

5. **What are the hardest classes?**

Shirt (6) and T-shirt/top (0) are often confused because they look visually similar.

8 Conclusion

In this practical, we implemented Fashion Clothing Classification using a Convolutional Neural Network (CNN) on the MNIST Fashion Dataset. The model architecture consisted of 3 convolutional layers with increasing filters (32, 64, 128), each followed by max pooling and dropout layers, a flatten layer, and dense layers with softmax output for 10-class classification. The data was preprocessed by normalizing pixel values to $[0, 1]$ and reshaping to 4D tensors. The model was compiled with Adam optimizer and sparse categorical cross-entropy loss, and trained for 10 epochs. A final test accuracy of approximately 90.31% was achieved, demonstrating the effectiveness of CNNs for image classification tasks.